Multithread Programming Analysis

Aistin Henry, Henry Glenn, and Zack Latta
Computer Science and Computer Engineering
Kansas State University
2900 College Ave, Manhattan, KS

Abstract

This project investigates parallel approaches for computing the maximum ASCII value for each line in a large Wikipedia text dataset. Three implementations were developed using pthreads, MPI, and OpenMP to evaluate different models of parallel execution. Each approach was tested on the Beocat cluster across a range of core counts, memory allocations, and node configurations. Results show that shared-memory approaches such as pthreads and OpenMP achieved strong performance on single-node systems, while MPI demonstrated effective scaling on a single node but degraded when distributed across multiple nodes due to communication overhead. Overall, performance improvements were primarily driven by increases in core count, while memory allocation had a lesser impact. These findings highlight the tradeoffs between shared-memory and distributed parallel programming models.

1. Introduction

For this project, we were tasked with finding the largest ASCII character value found in each line of a text document. The project description required that we print the largest ASCII value found for each line next to that line's index. The specific text document we were tasked to find these values for was a Wikipedia article dump file that contained one article per line. The project specification requires us to write three different C programs to complete this task, with each using a different library to interact with device threads and split up the work. One program must use pthreads, one program must use MPI, and one program must use OpenMP.

A multithreaded approach to this problem is more effective than a single-threaded approach because it allows for a fuller utilization of device I/O and memory resources as well as better separation of concerns. In this paper, we will first cover our implementations for the three separate solutions to the given task, then move on to how each of these solutions performed under different hardware configurations.

2. Implementation

2.1 Pthread Implementation

For our pthread solution program, we utilized a standard producer-consumer architecture where the main thread dealt with all I/O,

line parsing, and results printing operations, while a thread pool of workers managed the processing of all parsed file lines. The program starts by first reading a 1MB chunk of the Wikipedia dump file, identifying all the complete lines it contains, and putting them in a job batch. The job queue is then given a thread pool where each worker thread claims an available job, completes it, stores the result, and then claims another job to complete. While the thread pool is busy working through all the jobs in the newly submitted batch, the main thread reads another chunk from the Wikipedia dump file, such that when appended to the unused remainder of the previously read chunk, it totals to 1MB of data. This chunk is then parsed like the previous, and all complete lines are put in a new job batch. The main thread then waits for the thread pool to finish the current batch, after which it prints all the job results, then submits the new batch of jobs. The old batch of jobs is then destroyed, and the process repeats until the entirety of the Wikipedia dump file has been read and every line has been processed.

2.2 MPI Implementation

For our MPI solution program, we utilized a master-worker architecture. Rank 0 acted as the coordinator process and handled all file I/O, chunk preparation, and final output operations, while all remaining ranks acted as worker processes responsible for computing the maximum ASCII value of each assigned line. The program begins with rank 0 opening the Wikipedia dump file and reading a fixed number of lines into memory at a time. These lines are packed into a communication buffer and distributed to worker ranks using MPI point-to-point messaging. Each worker rank receives a batch of lines, computes the largest ASCII value present in each line, stores the results locally, and returns the completed values to rank 0. Once rank 0 receives the completed results from a worker, it outputs the line index and corresponding maximum ASCII value in the required order. Rank 0 then continues reading additional line batches and dispatching them until the full input file has been processed. After all work has completed, rank 0 sends termination messages to the worker ranks and the MPI environment is finalized. This implementation was selected because it separates file management from computation and allows processing to be distributed across multiple processes and multiple machines. Unlike pthreads, MPI does not rely on shared memory and can therefore scale across nodes in a cluster environment.

2.3 OpenMP Implementation

For the OpenMP solution program, we used the standard fork-join architecture, where the program will organize all the input data, then divide the tasks into different threads, and then join back together to finish the program. The program takes 3 inputs, the file name, the number of lines per chunk, and whether the program will print the highest value of each line. Once the program has organized the input data, it goes into the

threaded segment, where one thread will parse lines from the file into a 2d array, with the first dimension being the size of the chunk. Once the array has been filled, the program moves into a parallel for segment, where threads will pass a line into a helper method that will return the highest ASCII value from that line. The thread will then put the return value into the appropriate spot in a result array. Afterwards, the thread will go back and take the next available iteration and repeats the process. Once all the iterations have been handled, the program moves on to the next part, where one thread will print the highest ASCII value per line in the chunk if the user chose to have the program print them. Then the program loops back to the start where it will fill up a new chunk and repeats the process. This is done until the end of the file has been reached. After the program is done calculating the ASCII values, it joins back together to print the final report, which shows the number of lines read, the number of lines per chunk, and the execution time of the program. OpenMP relies on shared memory, so it cannot be run on multiple nodes. However if MPI is used with OpenMP, a program can be made where OpenMP can run on nodes with memory sharing between each node.

3. Evaluation

3.1 Pthread Evaluation

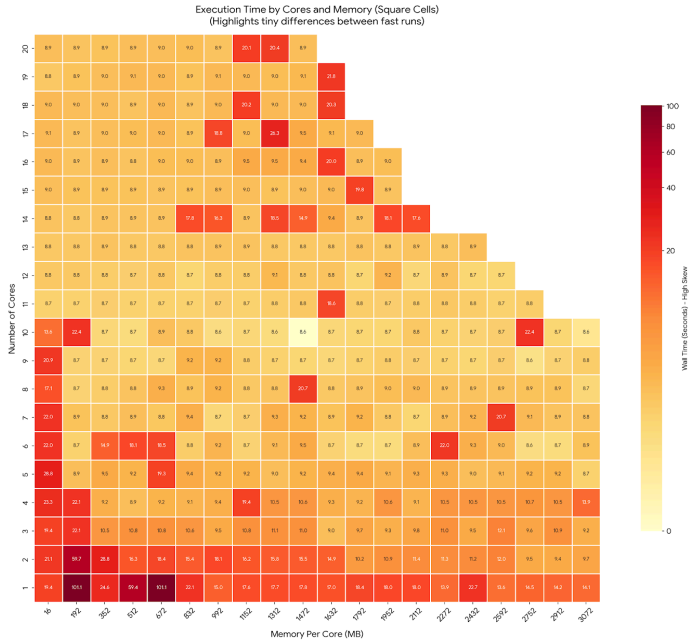

Figure 3.1.1: Pthread Execution Time

After testing our pthread solution program under a variety of conditions, we discovered that while increasing the number of cores and memory per core did improve execution speed, it only did so up to a certain point. This resource sweet spot is likely created as a result of the producer-consumer architecture the program utilizes. With a low number of cores, the shared resource lock for the current job batch is mostly available since a smaller number of threads need to access it. However, as the number of cores increase there becomes a point at which the shared resource lock is almost entirely unavailable, with one thread acquiring it immediately after another one releases it. As a result, increasing the number of cores after this point cannot improve the program's performance but rather only degrade it. While lower memory per core does appear to affect the program's performance, the number of cores clearly affects performance more significantly. This is proven by the fact that the results show even the lowest memory per core tested achieves near-optimal performance given enough cores. The same is not true for the lowest number of cores, even with the highest memory per core tested.

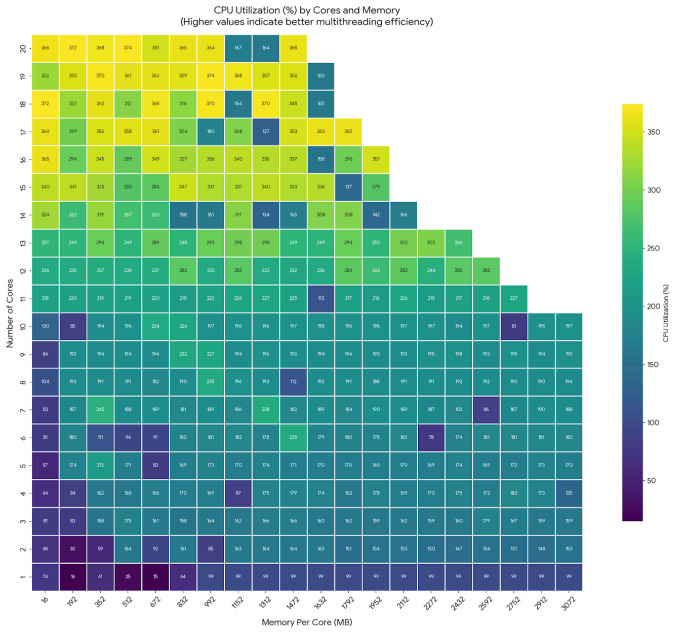

Figure 3.1.2: Pthread CPU Utilization

When looking at the CPU utilization of our pthread solution program, it is clear that the 1MB chunk reading limit is keeping our program from using the CPU to the fullest. What is likely happening is that the thread pool is finishing job batches before the main thread can read a new chunk and parse it into a batch. If a dynamic bulk reading system were in place that could change the reading chunk size depending on the number of CPU cores present, then CPU utilization would likely be consistently closer to the upper limit.

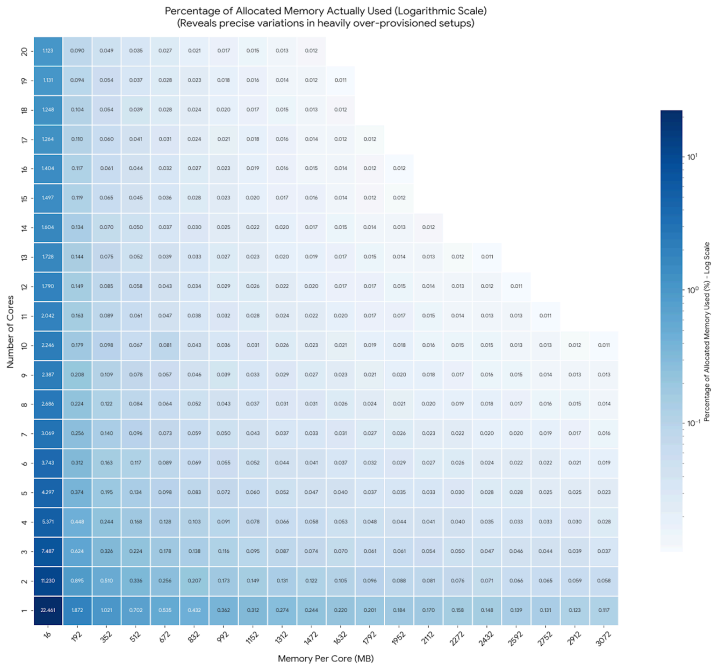

Figure 3.1.3: Pthread Memory Usage

Looking at memory usage, a similar trend can be seen. The 1MB chunk reading limit is once again the cause here. Since no dynamic system is in place as the amount of memory allocated is increased, the percent used by our program does not change, resulting in an overall decrease in memory usage.

3.2 MPI Evaluation

Next, we look at the MPI implementation. Table 1 summarizes the average runtime and standard deviation observed for each MPI configuration tested.

Configuration	Mean Time (s)	Std Dev (s)
1N-2R	24.10	2.39
1N-4R	4.84	3.43
1N-8R	2.74	0.04
2N-8R	11.29	0.15

Table 3.2.1: MPI Performance Summary

As shown in Table 1, the best average runtime was achieved using one node with eight MPI ranks. The two-node, eight-rank configuration performed substantially worse despite using the same total number of ranks, indicating that communication overhead significantly affected distributed execution performance. Representative MPI timing results are shown in Figures 3.2.2 and 3.2.3.

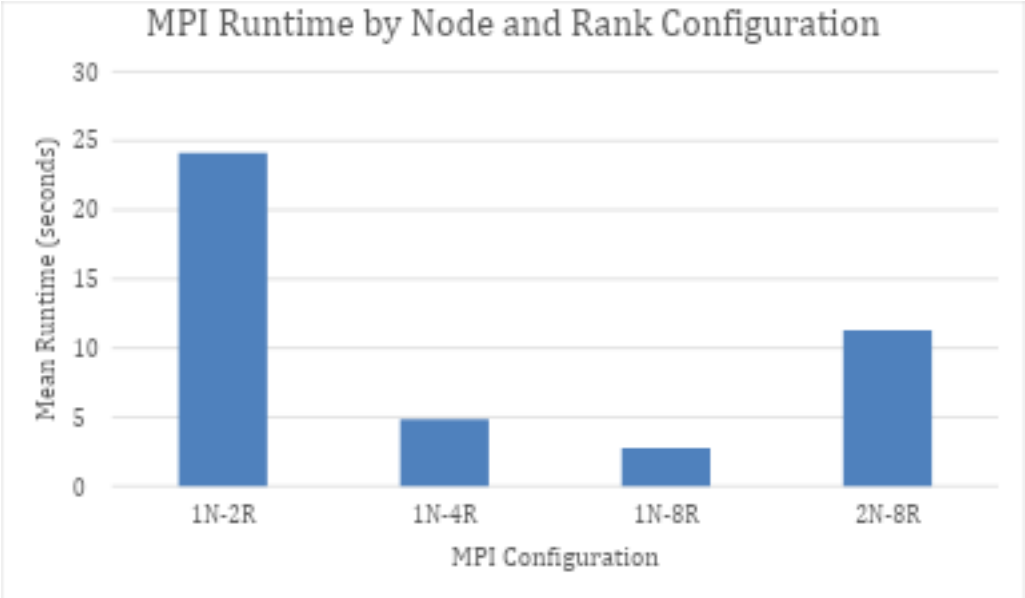

Figure 3.2.2: MPI Runtime

The MPI runtime results show substantial improvement as the number of ranks increased on a single node. Moving from 2 ranks to 4 ranks reduced

runtime significantly, while increasing to 8 ranks produced further gains. However, the improvement from 4 to 8 ranks was smaller, suggesting diminishing returns as communication and coordination overhead increased.

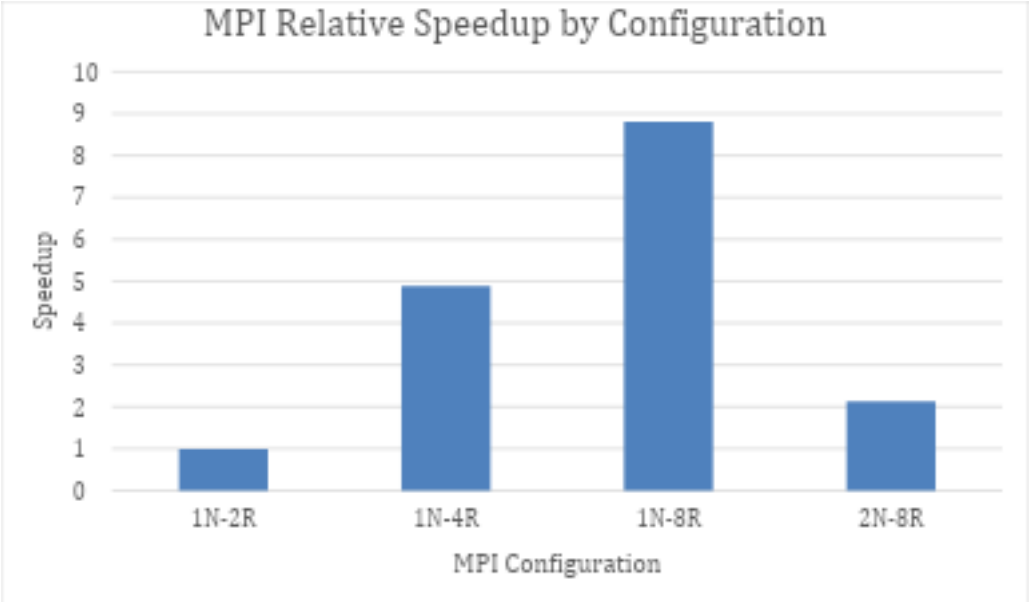

Figure 3.2.3: MPI Speedup

The speedup results confirm that increasing rank count improved performance within a single-node environment. However, the two-node configuration with eight total ranks performed worse than the single-node eight-rank configuration, indicating that inter-node communication overhead outweighed the benefit of distributing work across additional hardware. Compared to the pthread implementation, MPI provided strong scaling on a single node but was more sensitive to communication overhead when multiple nodes were used. The pthread solution benefited from shared memory and lower communication cost, while the MPI implementation offered portability to distributed systems at the cost of higher synchronization expense.

3.3 OpenMP Evaluation

Finally, the OpenMP implementation. Table 3.3.1 below shows a chart that measures the mean execution time and standard deviation for each configuration tested across 5 different jobs executions of the Slurm file. Figure 3.3.2 right below the table charts the information.

Configuration	Mean Runtime(sec)	St dev (sec)
1Cores- 64MB	12.1346	7.1363
1Cores- 128MB	11.1634	7.4318
1Cores- 256MB	11.4561	7.9518

1Cores- 512MB	10.4670	6.7331
1Cores- 1GB	10.1310	6.2943
1Cores- 1536MB	7.9266	3.2333
2Cores- 64MB	3.3876	0.1016
2Cores- 128MB	3.3068	0.0429
2Cores- 256MB	3.2054	0.0413
2Cores- 512MB	3.2093	0.0464
2Cores- 1GB	3.1824	0.0392
2Cores- 1536MB	3.1945	0.0417
4Cores- 64MB	2.2563	0.0408
4Cores- 128MB	2.1095	0.0302
4Cores- 256MB	2.1120	0.0301
4Cores- 512MB	2.1120	0.0328
4Cores- 1GB	2.1132	0.0281
4Cores- 1536MB	2.1097	0.0302
8Cores- 64MB	1.5725	0.0253
8Cores- 128MB	1.5717	0.0270
8Cores- 256MB	1.5762	0.0201
8Cores- 512MB	1.6048	0.0582
8Cores- 1GB	1.5865	0.0180
8Cores- 1536MB	1.5802	0.0187
16Cores- 64MB	1.3901	0.0128
16Cores- 128MB	1.3844	0.0245
16Cores- 256MB	1.3871	0.0135
16Cores- 512MB	1.3956	0.0165
16Cores- 1GB	1.3972	0.0410
16Cores- 1536MB	1.4020	0.0221
20Cores- 64MB	1.3277	0.0226
20Cores- 128MB	1.3152	0.0174
20Cores- 256MB	1.3145	0.0250
20Cores- 512MB	1.3140	0.0260
20Cores- 1GB	1.3091	0.0276
20Cores- 1536MB	1.3110	0.0300

Table 3.3.1: OpenMP Performance Summary

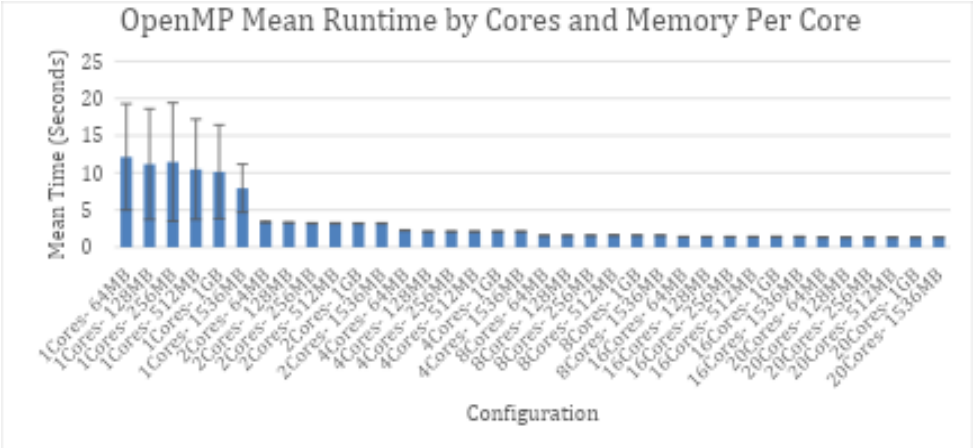

Figure 3.3.2: OpenMP Performance Chart

As shown in the chart, configurations utilizing one core take far longer and are much more volatile than the rest of the configurations. The reason this is the case is that the number of threads the program uses matches the number of cores being used, so configurations with one core cannot utilize the benefits of the fork-join architecture and the memory-sharing provided with OpenMP threading. For the configurations that have more than one core, there looks to be diminishing returns in the memory, as all the configurations sharing the same number of cores have very similar execution times, and the cause of the greatest difference in time change between the configurations is the number of cores. It is worth noting that the increase in cores has a diminishing return as well, with each increase in cores being used speeding up the program less. The fact that memory increase is negligible suggests that the program makes efficient use of the memory that it is given.

4. Conclusion

This project successfully evaluated a pthreads, MPI, and OpenMP implementation for computing the maximum ASCII character in each line of the Wikipedia dump file and revealed the performance tradeoffs of running those programs with different resource configurations. For the pthreads implementation, we found that it suffered from lock contention and chunk reading bottlenecks, making extra cores only improve performance up to a certain point. For the OpenMP implementation, we discovered that performance was heavily dependent on the number of cores provided to the process, with memory per core having very little effect. For the MPI implementation, we determined that it performed well on single-node systems but suffered severe performance degradation on multi-node systems, indicating that inter-node communication overhead far outweighed the extra resources the extra nodes provided. Ultimately, this project highlighted the fundamental tradeoffs between the shared-memory model and the scalable but communication-heavy nature of the distributed system model.